

APACHE DORIS 3.0.8

学习笔记

第七章：数据导入与实时写入

主线：导入不是文件复制，而是解析、转换、分片路由、副本写入、事务提交和版本发布；实时性来自合理微批，而不是逐行提交。

配套编号：notes-07

基准版本：Apache Doris 3.0.8

整理日期：2026-08

目录

目录	2
本章学习目标	4
7.1 Doris 导入体系与底层链路	4
一条记录经历什么	4
FE 与 BE 的分工	4
事务与版本可见性	4
实时仍需微批	5
7.2 Stream Load	5
网络链路	5
最小可用请求	5
参数背后的设计	5
返回值就是第一份诊断报告	6
四种关键状态	6
7.3 Broker Load 与 INSERT INTO SELECT	6
为何适合历史回灌	6
并行度的真实上限	6
和 TVF + INSERT SELECT 的选择	6
7.4 Routine Load : Kafka 持续消费	7
并发不是配置多少就有多少	7
微批次终止条件	7
Exactly-Once 的根本	7
状态与积压	7
7.5 Flink Doris Connector 与 CDC	7
Checkpoint 与 Doris 2PC	8
CDC 的三个不同问题	8
版本兼容	8
7.6 Group Commit : 高频小批优化	8
为何能减轻 Compaction	8
能力边界	9
7.7 数据转换与质量控制	9
临时列	9

Filtered 与 Unselected	9
严格模式与容忍度	9
7.8 Label、事务、幂等与两阶段提交	9
四个概念分工	9
确定性 Label	9
Stream Load 2PC	10
7.9 MemTable、Rowset、Segment 与 Compaction	10
为什么跨太多分区会提前 Flush	10
三类 Compaction	10
写放大与读放大的平衡	10
优化顺序	10
7.10 综合案例与导入故障诊断	11
电商实时数仓选型	11
症状到证据	11
最终方法论	11
第七章复习题	11
参考资料	12

本章学习目标

前六章说明了数据怎样组织、计算与加速。本章研究实时数仓的另一半：外部数据怎样持续、正确、高吞吐地形成 Doris 中可查询的新版本。

问题域	应能回答	核心对象
导入链路	一条外部记录怎样成为可见版本？	FE、BE、Tablet、事务
方式选型	Push、Pull、Kafka 和 Flink 怎么选？	Stream、Broker、Routine
一致性	网络重试为什么不重复，故障为什么不丢？	Label、Offset、2PC
写入性能	小批次为什么拖垮系统？	Group Commit、MemTable
长期稳定	写入快为何查询会逐渐变慢？	Rowset、Segment、Compaction
故障诊断	延迟、积压和脏数据如何定位？	Load 结果、Lag、Profile

一次导入的总链路



7.1 Doris 导入体系与底层链路

导入的产物不是原始文件，而是符合表模型、分区、分桶和列式存储规则的一组新 Rowset 版本。

一条记录经历什么

1. 解析 CSV、JSON、Parquet 等外部格式，得到输入字段。
2. 执行列映射、表达式、类型转换和质量检查。
3. 应用 Duplicate、Unique MOW 或 Aggregate 的表模型语义。
4. 根据分区键与分桶键计算目标 Tablet。
5. 在目标 BE 的 MemTable 中排序、聚合或去重，Flush 为 Segment。
6. 多数副本写入成功后提交事务，发布版本并变为可见。

FE 与 BE 的分工

组件	主要职责	为什么这样分
FE	认证、元数据、建事务、计划、提交、版本发布	保持为控制面，避免承载大数据流
BE	接收、解析、转换、路由、写 MemTable/Rowset	数据面可随节点并行扩展

事务与版本可见性

PREPARE -> LOADING -> COMMITTED -> VISIBLE
 \-> ABORTED

COMMITTED = committed, not query-visible yet
 VISIBLE = version published and query-visible

原子性避免百万行批次只暴露其中六十万行。一次导入要么整体发布，要么失败回滚；查询不会看到半批次。

实时仍需微批

visible_latency $\approx$ batch_wait + network + parse + BE_write + publish
 one_row_per_txn -> high fixed cost + tiny Rowsets
 one_batch_per_s -> stable throughput with bounded freshness

7.2 Stream Load

Stream Load 是客户端通过 HTTP 主动 Push 一个批次，Doris 同步返回该隐式事务的结果。

网络链路

FE 不承载主要数据流



最小可用请求

```

curl --location-trusted -u user:password \
  -H 'Expect:100-continue' \
  -H 'label:orders_20260818_0001' \
  -H 'column_separator:,' \
  -H 'columns:order_id,user_id,status,amount,event_time' \
  -H 'strict_mode:true' -H 'max_filter_ratio:0.01' \
  -T orders.csv \
  http://fe1:8030/api/demo/orders/_stream_load
  
```

参数背后的设计

- --location-trusted：跟随 FE 到 BE 的重定向并携带认证。
- Expect:100-continue：先校验头部，再发送大文件，减少无效传输。
- label：为业务批次提供稳定身份，网络不确定时用原 Label 重试。
- strict_mode：决定类型异常怎样判定；max_filter_ratio 决定整批容忍度。

返回值就是第一份诊断报告

指标	高值优先怀疑
BeginTxnTimeMs	FE 事务并发或排队
StreamLoadPutTimeMs	规划、分区/Tablet 元数据压力
ReadDataTimeMs	客户端、网络或格式解析
WriteDataTimeMs	BE、磁盘、热点 Tablet、副本和 Compaction
CommitAndPublishTimeMs	版本发布、慢副本和小事务积压

四种关键状态

- Success：事务成功并完成正常发布。
- Publish Timeout：事务通常已提交，不能换新 Label 盲目重导；应查询原事务状态。
- Label Already Exists：检查 ExistingJobStatus，FINISHED 可视为原批次已成功。
- Fail：结合 Message、FilteredRows 和 ErrorURL 定位。

7.3 Broker Load 与 INSERT INTO SELECT

Stream Load 由客户端 Push；Broker Load 只提交远程文件位置，由 Doris 从 S3/HDFS 并行 Pull。

为何适合历史回灌

TB 级文件已在对象存储时，让单客户端再次读取并上传会把网络、分片和重试都压在一处。Broker Load 由 FE 枚举文件并把读取任务分配给多个 BE，更容易利用集群并行度。

```
LOAD LABEL demo.orders_backfill_202608 (
  DATA INFILE('s3://warehouse/orders/2026/08/*.parquet')
  INTO TABLE orders FORMAT AS 'parquet'
)
WITH S3 ('s3.endpoint'='https://s3.example.com', ...)
PROPERTIES ('timeout'='14400', 'strict_mode'='true');
```

```
SHOW LOAD FROM demo WHERE LABEL='orders_backfill_202608';
```

并行度的真实上限

```
parallelism ~ file_count * splittability * available_BEs * tablet_spread
one 2-TB gzip      -> hard to split, single-reader bottleneck
200 Parquet files -> easier distributed parallel reads
```

和 TVF + INSERT SELECT 的选择

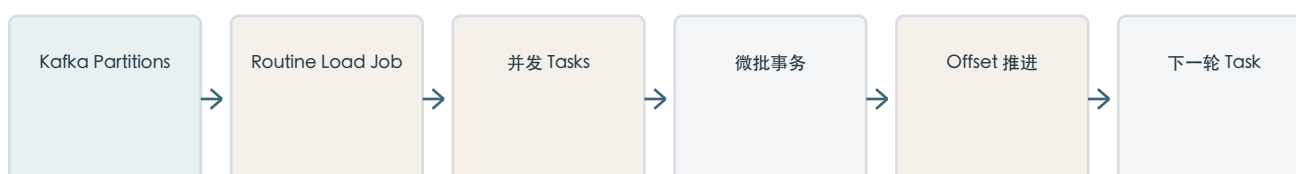
维度	Broker Load	INSERT SELECT + TVF
执行	异步 Load Job	通常同步 SQL
转换	导入映射表达式	完整 SELECT、JOIN、CAST、函数
预览	不方便	可先 SELECT 外部文件
场景	大规模、可追踪回灌	中等规模或复杂 SQL 转换

一个 Broker Load 还可包含多个 DATA INFILE 描述，并在同一导入事务内原子提交多张目标表；普通多个 Stream Load 请求不会自动获得这种整体边界。

7.4 Routine Load : Kafka 持续消费

Routine Load 是 Doris 内部的常驻 Kafka 消费 Job；Job 不断生成短生命周期 Task，每个 Task 把一个微批次写成事务。

Job、Task、Transaction、Offset



并发不是配置多少就有多少

```
actual_tasks = min(kafka_partitions, desired_concurrent_number, system_limit)
partitions=3, desired=30 -> actual_tasks <= 3
```

微批次终止条件

max_batch_interval、max_batch_rows、max_batch_size 任意一个先达到就提交。调小可降低可见延迟，却增加事务、版本和 Compaction；调大提高吞吐，但增加延迟与失败重试范围。

Exactly-Once 的根本

```
consume offsets 100-199
-> write Doris transaction T1
-> commit T1
-> persist next_offset=200
```

T1 fails -> offset does not advance -> retry from 100

状态与积压

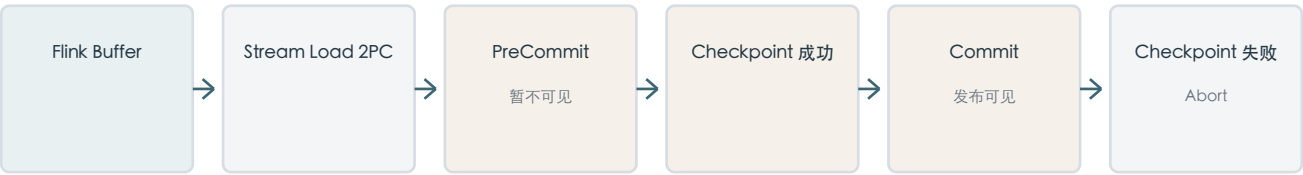
状态/现象	含义与处理
RUNNING	正常调度，但仍需观察 Lag 和实际吞吐
PAUSED	可恢复；先处理脏数据、认证、Offset 等根因
STOPPED	作业已结束，不能简单 RESUME
Lag 上升	依次看 Job、Partition 并发、BE 写入、热点和 Compaction

7.5 Flink Doris Connector 与 CDC

Routine Load 适合 Kafka 简单映射；MySQL CDC、多流 Join、窗口与复杂状态处理更适合 Flink。Connector 的底层写入仍然依赖 Stream Load。

Checkpoint 与 Doris 2PC

端到端提交边界



若数据已经写入 Doris 而 Checkpoint 尚未成功，Flink 故障恢复会从旧 Checkpoint 重放。2PC 将数据准备与最终可见拆开，只有全局 Checkpoint 成功后才 Commit；失败则 Abort。

CDC 的三个不同问题

问题	对应机制	不要混淆
故障重放	Checkpoint + Stream Load 2PC	保证同一批不重复发布
主键更新	Unique Key MOW	维护同一主键最新值
业务乱序	Sequence Column	让版本更大的事件获胜

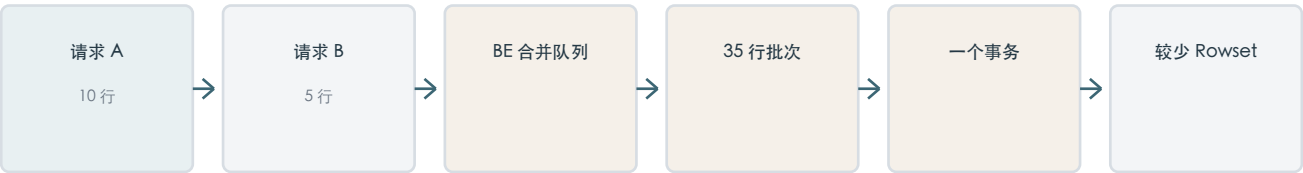
版本兼容

Connector 必须与 Flink 和 Doris 3.0.8 的兼容矩阵匹配；sink.label-prefix 要稳定且在不同作业间唯一。不能把最新 Connector 的参数直接当成 3.0.8 固有行为。

7.6 Group Commit：高频小批优化

Group Commit 把多个客户端小请求在 BE 侧合并为较大的事务，用少量可见延迟换取更少事务、版本与小 Rowset。

服务器端攒批



模式	返回时机	可见性与场景
off_mode	普通导入完成	不使用 Group Commit
sync_mode	合并事务提交并可见后	返回后可查，适合明确可见性
async_mode	数据写入 BE WAL 后	稍后可见，适合低延迟高频写

为何能减轻 Compaction

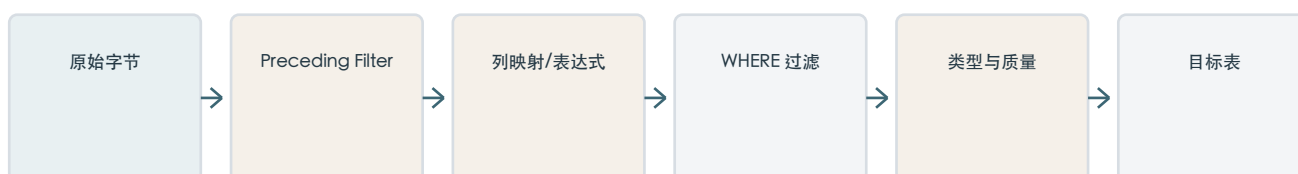
100 requests/s -> 100 txns/s -> many tiny versions and Rowsets
 Group Commit -> fewer txns -> fewer versions -> less merge work

能力边界

显式事务、Stream Load 2PC、独立 Label 边界、已足够大的批次或严格提交顺序，不应强行使用 Group Commit。它优化的是可合并的小请求，不是替代所有导入事务。

7.7 数据转换与质量控制

转换顺序决定可见字段语义



临时列

input amount in cents: 9950
 columns: order_id,user_id,tmp_amount,amount=tmp_amount/100
 tmp_amount is an import-only temporary column

Filtered 与 Unselected

类型	来源	是否计入错误率
Filtered Rows	类型失败、超长、NULL、无分区等质量问题	是
Unselected Rows	WHERE 等业务规则主动排除	否

严格模式与容忍度

strict_mode 决定哪些转换被判为异常，max_filter_ratio 决定整个批次最多容忍多少异常行。核心事实表应严格；日志原始层可以极小容忍，但必须保存错误样例、监控趋势并允许回放。把比例设为 1 再忽略错误，会把数据质量事故伪装成成功。

7.8 Label、事务、幂等与两阶段提交

四个概念分工

概念	解决的问题
原子性	一个批次全部可见或全部不可见
Label 幂等	同一数据库内，同批次重试不会成功两次
上游重试	失败后继续尝试，防止丢失
Exactly-Once	至少一次重试 + 固定身份去重 + 正确提交边界
2PC	让 Doris 的最终可见服从外部协调者决定

确定性 Label

```
stable labels:
orders_kafka_p3_offset100_199
    flink_orders_checkpoint_1024
```

wrong: create a new UUID on every retry
result: an uncertain retry becomes a new batch

Label 有数据库作用域和保留周期，不是永久去重数据库。业务重试窗口要与保留窗口匹配。

Stream Load 2PC

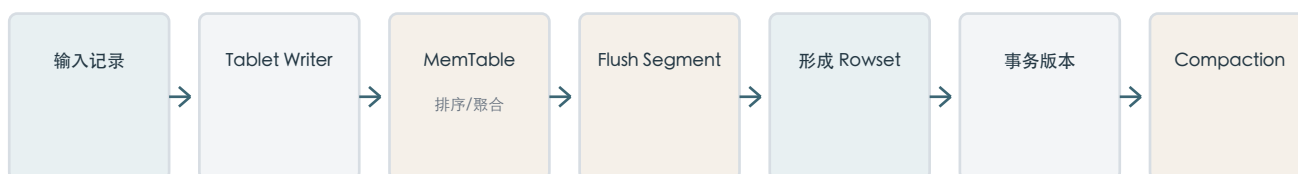
```
phase 1: two_phase_commit:true -> PRECOMMITTED (not visible)
phase 2: txn_operation:commit -> publish
         txn_operation:abort  -> discard
```

2PC 会延长事务生命周期并占用 FE/BE 资源，只应在 Flink Checkpoint 或外部事务协调场景使用；普通文件重试通常用固定 Label 即可。

7.9 MemTable、Rowset、Segment 与 Compaction

Doris 采用追加新版本、后台合并的思路：写入避免原地随机修改，但必须用 Compaction 控制长期读放大。

从内存批次到长期存储



为什么跨太多分区会提前 Flush

365 partitions * 32 buckets = 11,680 potential Tablet Writers
many writers -> fragmented MemTables -> early Flush -> tiny Segments

三类 Compaction

类型	主要合并对象	解决什么
Segment Compaction	同一 Rowset 内大量 Segment	控制单次大导入内部文件数
Cumulative	较新的小 Rowset	快速降低增量版本和多路归并
Base	历史 Base 与累计结果	长期回收旧版本并降低读放大

写放大与读放大的平衡

永不合并会让查询打开大量 Rowset；每次增量都重写全部历史又会产生巨大写放大。Cumulative 和 Base 分层合并，让系统在版本数、读放大、后台 IO 和写放大之间折中。

优化顺序

先检查批次过小、Group Commit、跨分区范围、桶数、热点 Tablet、Kafka/Flink 并发、文件碎片和磁盘资源，最后才调整 Compaction 线程。不能只靠加快清理速度掩盖上游持续制造小文件。

7.10 综合案例与导入故障诊断

电商实时数仓选型

数据源	推荐链路	解决重点
MySQL 订单 CDC	Flink CDC + 2PC + Unique MOW + Sequence	更新、重放、乱序
Kafka 行为日志	Routine Load + Duplicate Key	持续消费、秒级可见
S3 历史 Parquet	Broker Load，按日期分批	TB 级并行回灌
应用高频事件	Stream/INSERT + Group Commit	减少小事务和 Rowset

症状到证据

症状	优先检查
Stream Load 超时	原 Label、阶段耗时、事务状态；不要换 Label 盲目重试
Success 但行数不对	Loaded/Filtered/Unselected、Unique 覆盖、Aggregate 聚合
Routine 不消费	JobState、Reason、认证、Offset、质量、Task Slot
Kafka Lag 上升	Job -> 实际并发 -> BE 写入 -> 热点/Compaction
Flink Checkpoint 变慢	PreCommit、BE 写入、2PC 积压、Label 前缀与版本
导入快但查询变慢	事务频率、Rowset、Compaction Score、磁盘 IO
数据重复	新 Label 重试、2PC、Label 保留、表模型和多写入源

最终方法论

先按数据位置、方向、批量和事务边界选择导入方式；再用确定性身份保证重试；最后从 MemTable、版本与 Compaction 评估长期稳定性。

第七章复习题

建议用一句话结论、底层链路、设计取舍和诊断证据四层回答。

1. 为什么 Doris 导入不是简单地把文件复制到 BE？
2. FE 和 BE 在导入链路中分别负责什么？
3. Stream Load 与 Broker Load 的数据流方向有什么不同？
4. 为什么 Broker Load 更适合 S3/HDFS 上的大规模历史回灌？
5. Broker Load 和 INSERT INTO SELECT + TVF 应怎样选择？
6. Broker Load 的并行度受到哪些因素限制？
7. Routine Load 中 Job、Task、Transaction 和 Offset 分别是什么？
8. 为什么 desired_concurrent_number 不等于实际消费并发？
9. 三个 max_batch 参数如何共同决定微批次？
10. Routine Load 怎样把 Kafka Offset 与 Doris 事务绑定，实现 Exactly-Once？
11. Routine Load 的 PAUSED 和 STOPPED 有什么区别？

12. Flink Doris Connector 为什么仍然依赖 Stream Load ?
13. Flink Checkpoint 与 Doris Stream Load 2PC 怎样配合 ?
14. 2PC 与 Sequence Column 分别解决什么问题 ?
15. Group Commit 为什么能同时降低 FE 压力和 Compaction 压力 ?
16. sync_mode 与 async_mode 的返回时机和可见性有何区别 ?
17. Filtered Rows 和 Unselected Rows 有什么区别 ?
18. Label、事务原子性、幂等和 Exactly-Once 分别解决什么问题 ?
19. Rowset、Segment、Cumulative Compaction 和 Base Compaction 是什么关系 ?
20. 如果导入正常但查询越来越慢，应按照什么顺序排查 ?

参考资料

本文以 Apache Doris 3.0.8 为基准。3.x 文档仅用于核对与 3.0.8 兼容的能力，后续版本新增参数不纳入正文结论。

- [1] Doris 3.x - Load Overview
- [2] Doris 3.0 - Stream Load
- [3] Doris 3.x - Broker Load
- [4] Doris 3.x - Routine Load
- [5] Doris 3.x - Flink Doris Connector
- [6] Doris 3.x - Group Commit
- [7] Doris 3.x - Transactions
- [8] Doris 3.0 - Compaction Status
- [9] Release 3.0.8